

AI 2002: Artificial Intelligence

Local Search

Spring 2026

[These slides (mostly) are based on those created by Dan Klein and Pieter Abbeel, with some slides were updated by Stuart Russell and Dawn Song (ai.berkeley.edu).]

[Some slides are by Mr. Sandesh Kumar, and Ms. Ansum Hamid, FAST-NUCES, Karahi]

Local Search

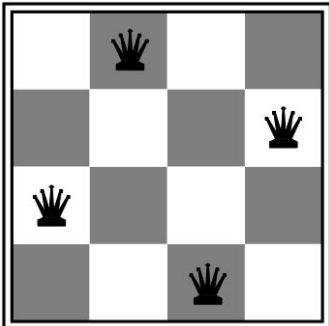


Local Search

- Operate using a single current node and generally move only to neighbors of that node.
- ease up on completeness and optimality in the interest of improving time and space complexity?
- Although local search algorithms are not systematic, they have two key advantages:
 - They use **very little memory** (usually a constant amount), and
 - They can often find reasonable solutions in large or **infinite** (continuous) state spaces.
- In addition to finding goals, local search algorithms are useful for solving pure optimization problems, in which the aim is to find the best state according to an objective function.

Local search algorithms

- In **many** optimization problems, **path** is irrelevant; the goal state **is** the solution
- In some optimization problems, the **goal is not known**, and the aim is to find the **best state**.
- Then state space = set of “complete” configurations;
find **configuration satisfying constraints**, e.g., n-queens problem



- In such cases, can use **iterative improvement** algorithms: keep a single “current” state, try to improve it
- Constant space, suitable for online as well as offline search

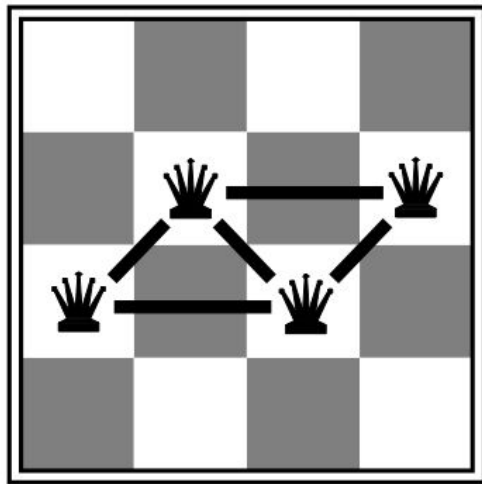
Hill Climbing

- Simple, general idea:
 - Start **wherever**
 - **Repeat**: move to the best neighboring state
 - If **no neighbors better** than current, **quit**

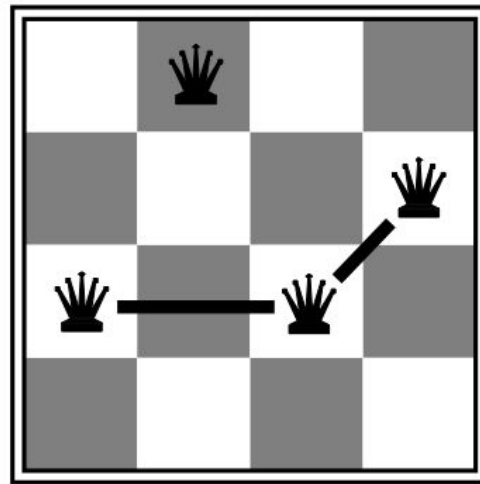
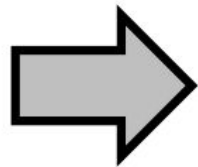


Hill climbing example - n -queens problem

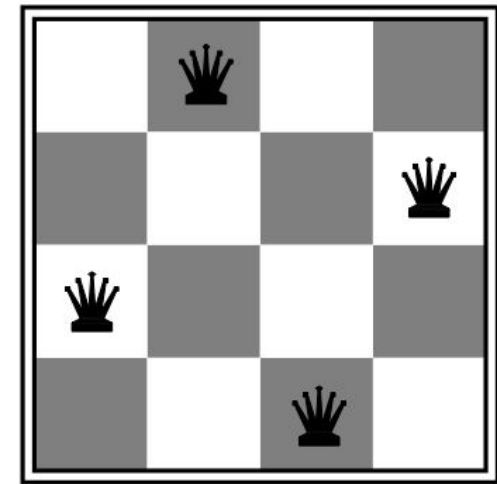
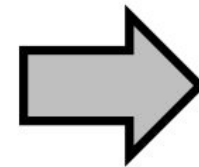
- Goal: n queens on board with no **conflicts**, i.e., no queen attacking another
- States: n queens on board, one per column
- Actions: move a queen in its column
- Heuristic value function: number of conflicts



$h = 5$



$h = 2$



$h = 0$

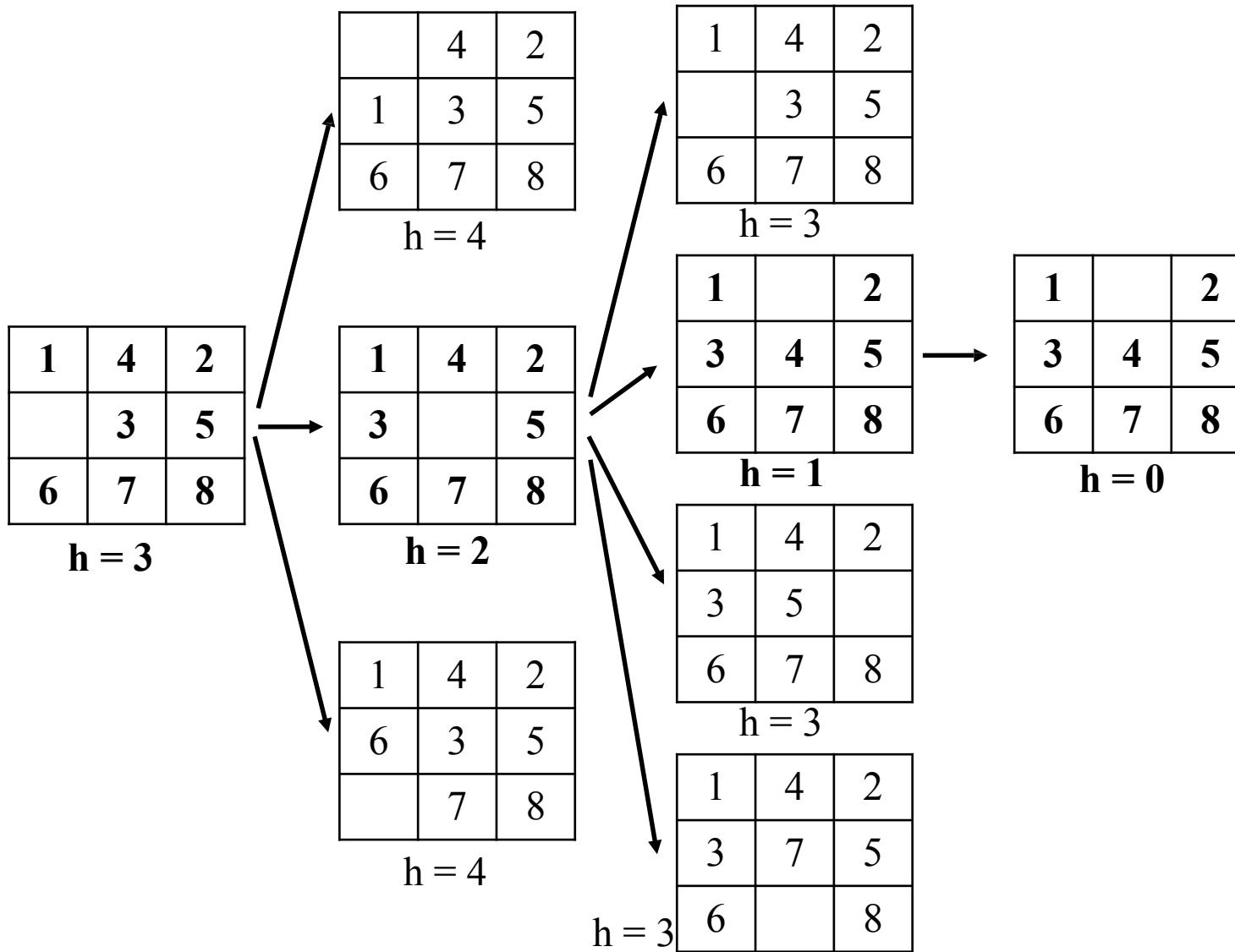
Hill-climbing algorithm (Greedy local search)

```
function HILL-CLIMBING(problem) returns a state
  current ← make-node(problem.initial-state)
  loop do
    neighbor ← a highest-valued successor of current
    if neighbor.value ≤ current.value then
      return current.state
    current ← neighbor
```

“Like climbing Everest in thick fog with amnesia”

Hill Climbing Example

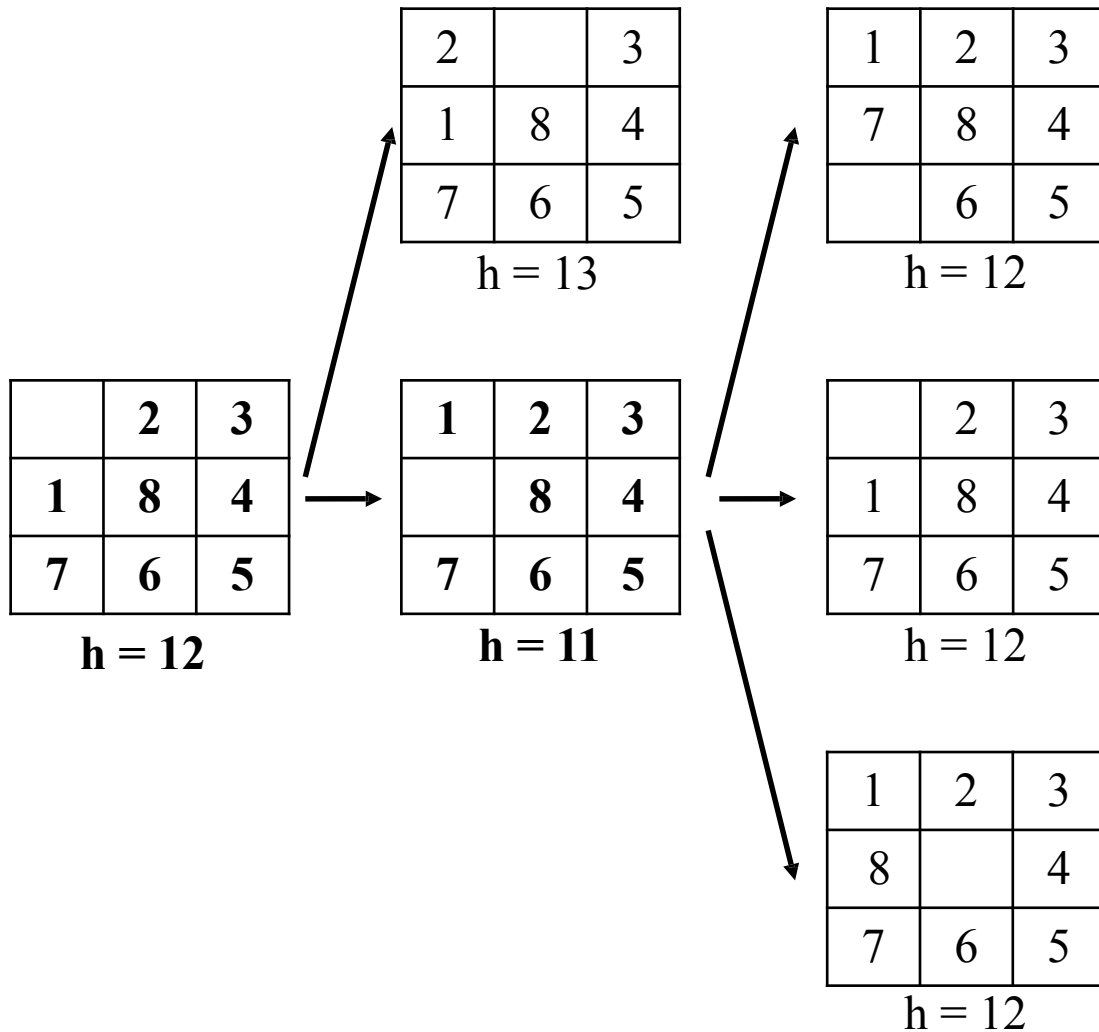
8-puzzle: a solution case



*Heuristic function is
Manhattan
Distance*

Hill Climbing Example

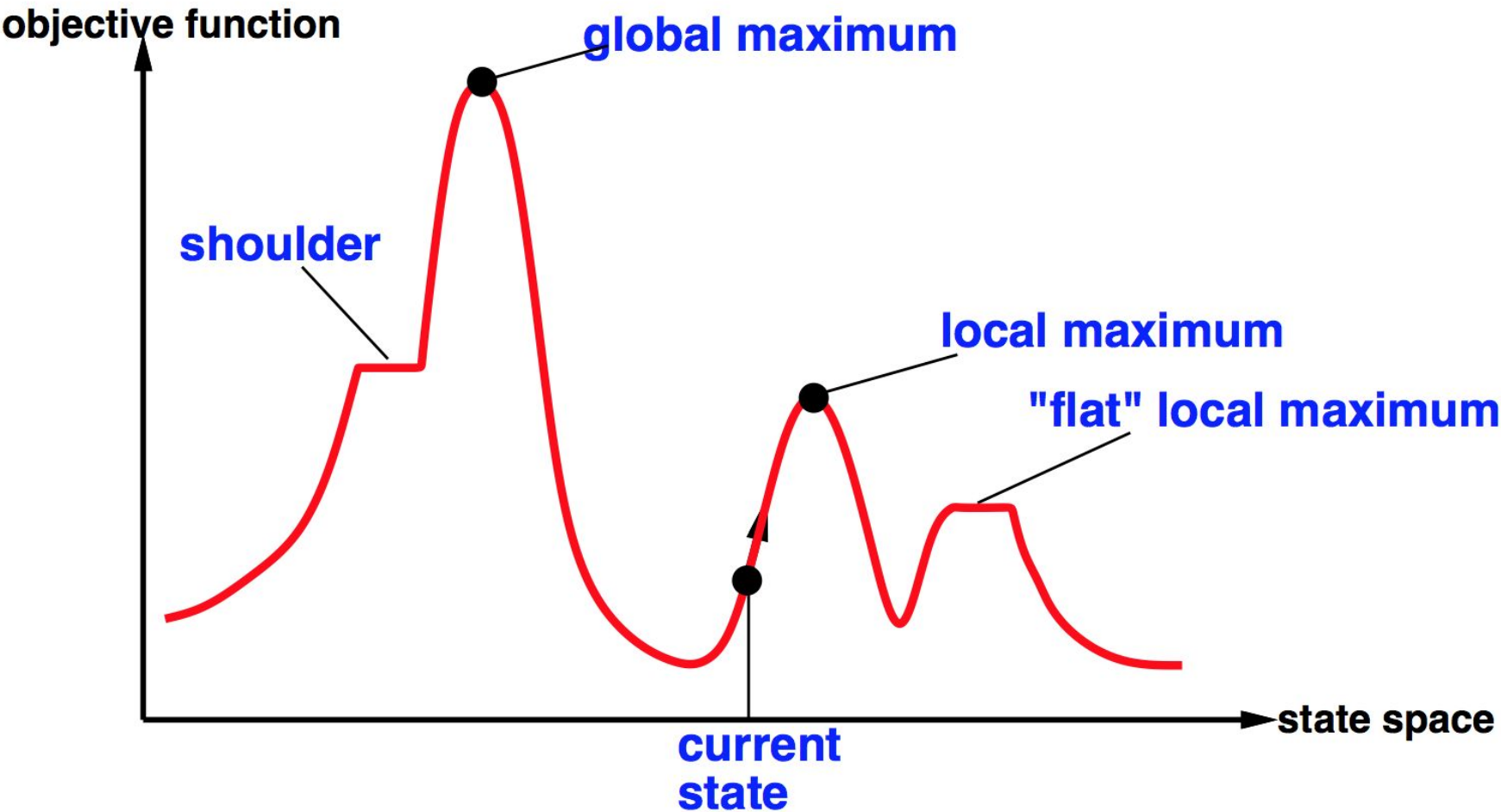
8-puzzle: stuck at local maximum



*Heuristic function is
Manhattan
Distance*

We are stuck with a local maximum.

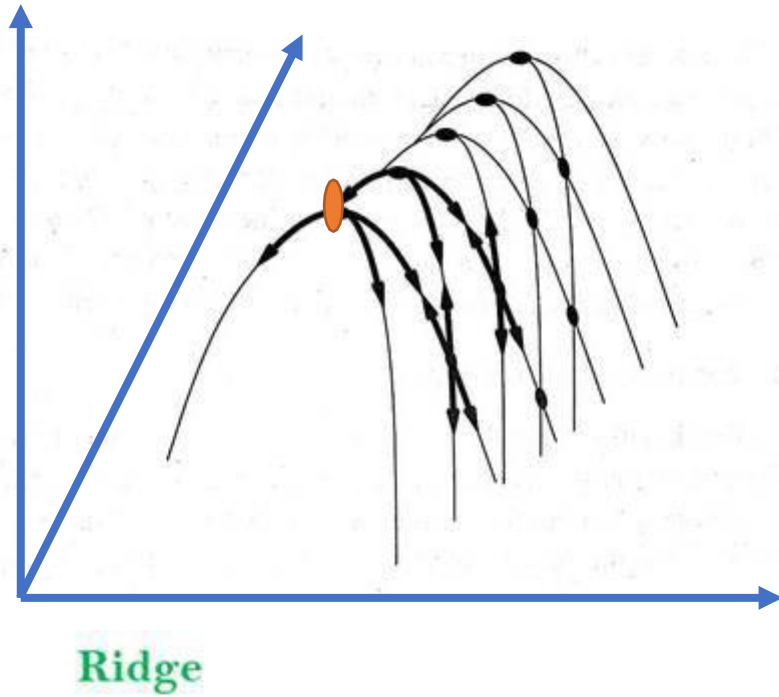
Global and local maxima



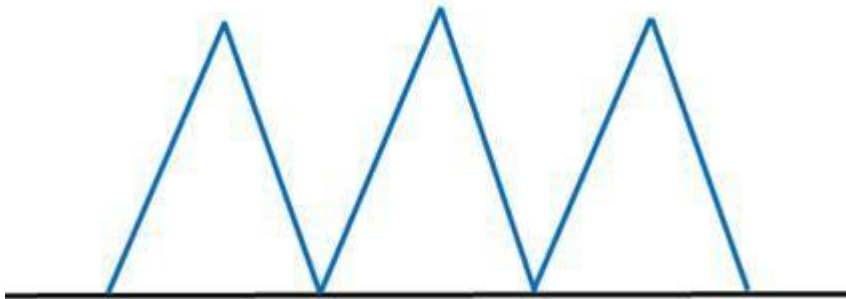
Plateau (Flat area):

- Flat local maximum
- Shoulder

Problems in Hill Climbing...



3. Ridges: A ridge is a special form of the local maximum. It has an area which is higher than its surrounding areas, but itself has a slope, and cannot be reached in a single move.



Hill Climbing Example

n-queens

- Starting from a randomly generated 8-queens state,
 - steepest-ascent hill climbing gets stuck 86% of the time,
 - solving only 14% of problem instances.
- The Hill Climbing algorithm halts if it reaches a plateau.
 - One possible solution is to allow some sideways move in the hope that the plateau is really a shoulder.
 - For example, we could allow up to 100 consecutive sideways moves in the 8-queens problem. This raises the percentage of problem instances solved by hill climbing from 14% to 94%.

Hill Climbing

- Hill Climbing is NOT complete.
- Hill Climbing is NOT optimal.
- Why use local search?
 - **Low memory** requirements – usually constant
 - Effective – Can often find good solutions **in extremely large** state spaces
 - **Randomized variants** of hill climbing can solve many of the drawbacks in practice.
- Many variants of hill climbing have been invented.

Simulated Annealing

- Instead of picking the best move, Simulated Annealing picks a random move.
 - If the move improves the situation, it is always accepted.
 - Otherwise, the algorithm accepts the move with some probability less than 1.
- The probability decreases exponentially with the “badness” of the move—the amount ΔE by which the evaluation is worsened.
- The probability also decreases as the “temperature” T goes down: “bad” moves are more likely to be allowed at the start when T is high, and they become more unlikely as T decreases.
 - If the schedule lowers T slowly enough, the algorithm will find a best state with probability approaching 1.
- Simulated Annealing is widely used in VLSI layout and airline scheduling.

Simulated Annealing

- A hill-climbing algorithm is **incomplete**
- A purely random walk is **complete** but extremely inefficient.
- Idea: Combine both for efficiency and completeness.
- Idea implementation: escape local maxima by allowing some “bad” moves but gradually decrease their size and frequency.
- The simulated annealing algorithm, a version of **stochastic hill climbing** where some downhill moves are allowed.
- **Annealing: the process of gradually cooling metal to allow it to form stronger crystalline structures**

Simulated annealing algorithm

function SIMULATED-ANNEALING(problem,schedule) **returns** a state

current $\leftarrow$ problem.initial-state

for t = 1 **to** ∞ **do**

 T $\leftarrow$ schedule(t)

if T = 0 **then return** current

 next $\leftarrow$ a randomly selected successor of current

$\Delta E \leftarrow$ next.value – current.value

if $\Delta E > 0$ **then** current $\leftarrow$ next

else current $\leftarrow$ next only with probability $e^{\Delta E/T}$

$$e^{(-1/100)} = 0.99$$

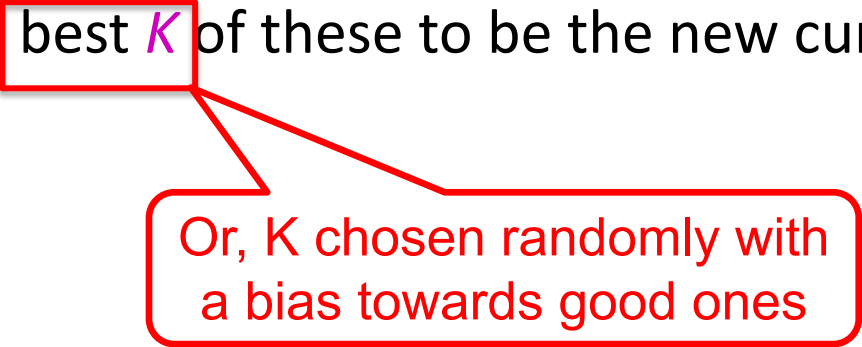
$$e^{(-1/10)} = 0.90$$

$$e^{(-1/1)} = 0.36$$

$$e^{(-1/0.1)} = 0.000045$$

Local beam search

- Basic idea:
 - K copies of a local search algorithm, initialized randomly
 - For each iteration
 - Generate ALL successors from K current states
 - Choose best K of these to be the new current states

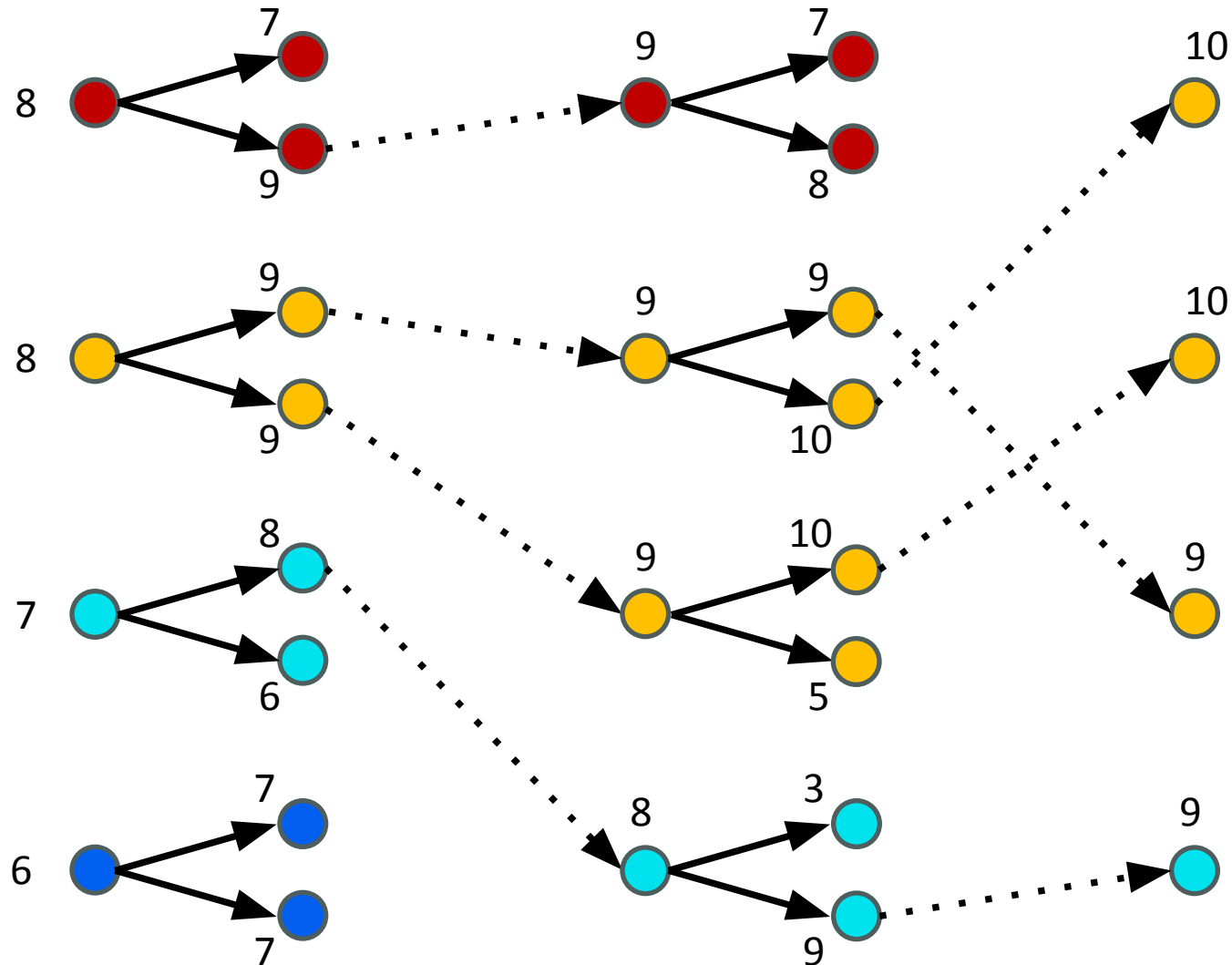


Or, K chosen randomly with
a bias towards good ones

Local beam search

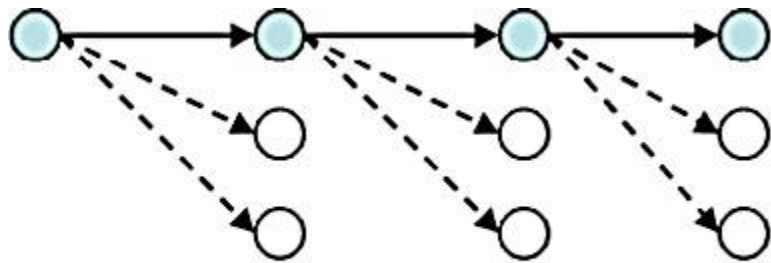
- keeps track of '**K**' states rather than just one.
 - It begins with k randomly generated states.
 - At each step, all the successors of all k states are generated.
 - If any one is a goal, the algorithm halts.
 - Otherwise, it selects k best successors from the complete list and repeats.
- Not the same as '**K**' searches run in parallel!
 - In a local beam search, searches that find good states recruit other searches to join them.
 - In a random-restart search, each search process runs independently of the others.

Beam search example ($K=4$)

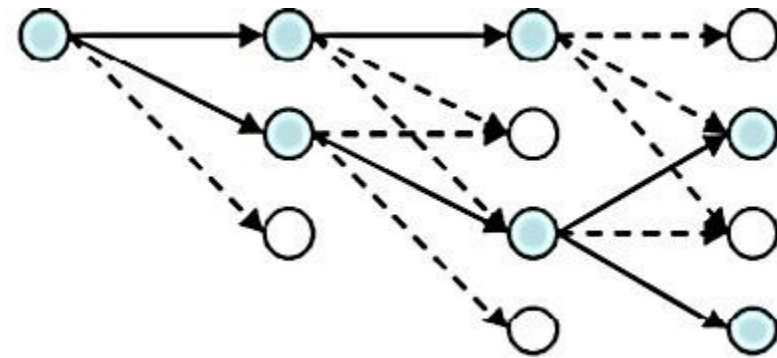


Local Beam Search

- Start with k randomly generated states ($\beta = 2$ or 3)
- At each iteration, all the successors of all k states are generated
- If any one is a goal state, stop; else select the k best successors from the complete list and successors from the complete list and repeat



Greedy Local



Local Beam

Stochastic Beam Search

- **Problem:** quite often, all k states end up on same local hill in local beam search algorithm
- **Idea:** choose k successors randomly, biased towards good ones
- Instead of choosing the best k from the pool of candidate successors, stochastic beam search chooses k successors at random, with the probability of choosing a given successor being an increasing function of its value.
- Stochastic beam search bears some resemblance to the process of natural selection, whereby the “successors” (offspring) of a “state” (organism) populate the next generation according to its “value” (fitness).

Genetic Algorithms (GAs)

- A variant of stochastic beam search in which successor states are generated by combining two parent states rather than by modifying a single state.
- Like beam searches, GAs begin with a set of k randomly generated states, called population.
- Encoding
 - Each state is represented as a string over finite alphabet (chromosome)—mostly, a string of 0s and 1s.
 - An 8-queens state must specify the positions of 8 queens, each in a column of 8 squares, and so it requires $8 \times \log_2(8) = 24$ bits.
 - Alternatively, the state could be represented as 8 digits, each in the range from 1 to 8. (we use this).
- Each state is rated by an objective function, or the fitness function (bigger number is better).
- Pairs of individuals are randomly selected for reproduction, with some probability.
- Crossover
 - From each pair, new offsprings (children) are created by crossing over parent strings at the crossover point.
 - A crossover point is chosen randomly from the positions in the string.
- Each child is subject to random mutation with a small independent probability.

Computational Model

Step 1: Representing individuals.

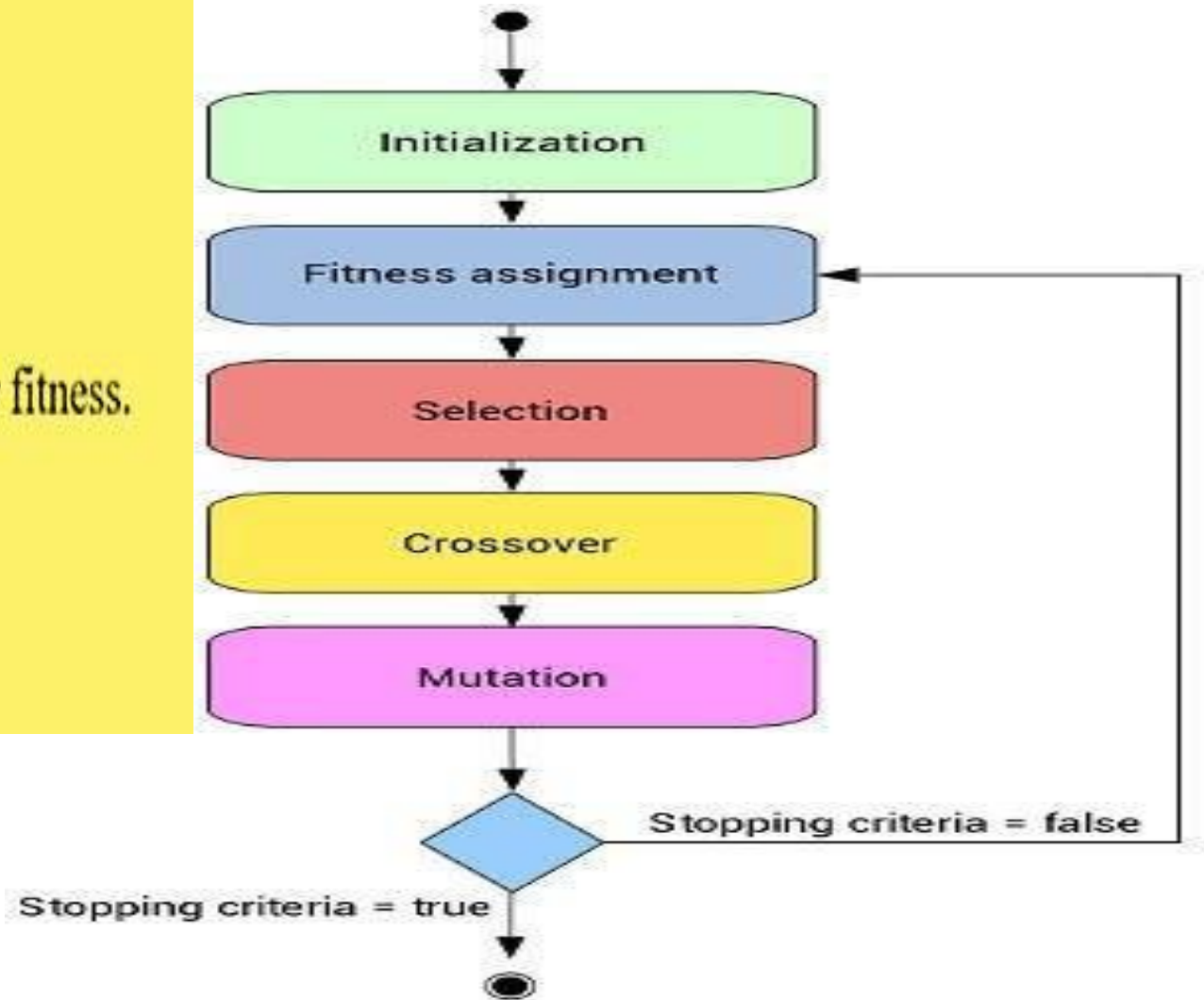
Step 2: Generating an initial Population.

Step 3: Applying a Fitness Function.

Step 4: Selecting parents for mating in accordance to their fitness.

Step 5: Crossover of parents to produce new generation.

Step 6: Mutation of new generation to bring diversity.



Encoding

*The process of representing the solution in the form of a **string** that conveys the necessary information.*

- **Binary Encoding** – Most common method of encoding. Chromosomes are strings of 1s and 0s and each position in the chromosome represents a particular characteristic of the problem.

Permutation Encoding – Useful in ordering problems such as the Traveling Salesman Problem (TSP). Example. In TSP, every chromosome is a string of numbers, each of which represents a city to be visited.

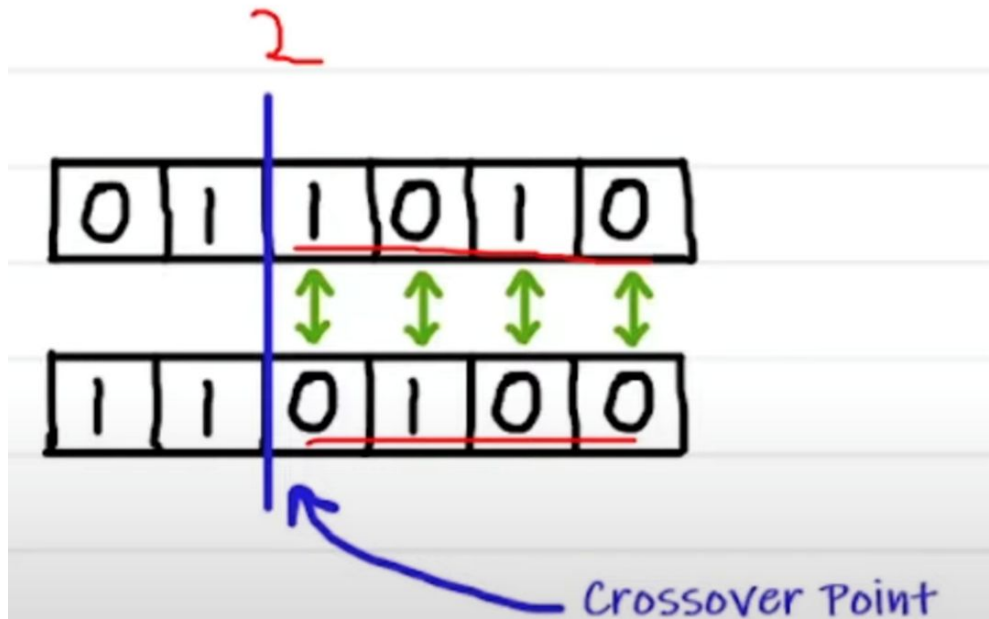
- **Value Encoding** – Used in problems where complicated values, such as real numbers, are used and where binary encoding would not suffice.

Crossover

It is the process in which two chromosomes (strings) combine their genetic material (bits) to produce a new offspring which possesses both their characteristics.

- Two strings are picked from the mating pool at random to cross over.*
- The method chosen depends on the Encoding Method.*

Single Point Crossover



Offsprings

O1: `0 1 0 1 0 0`

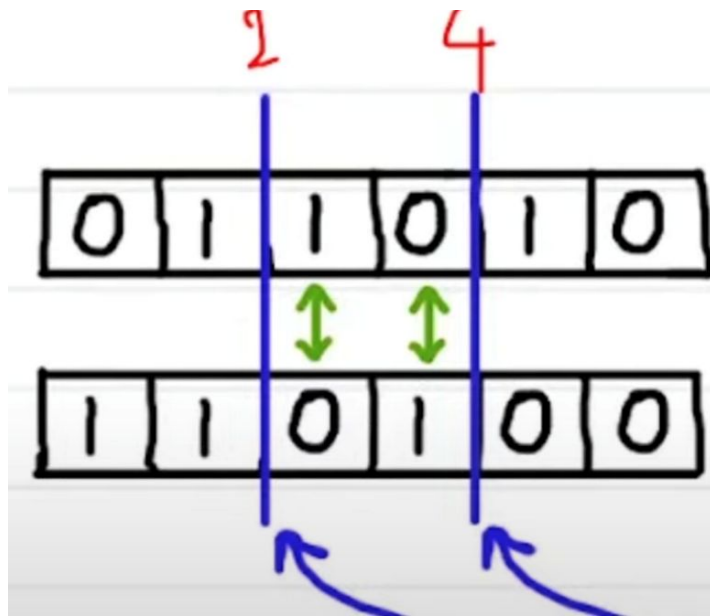
O2: `1 1 1 0 1 0`

Crossover

- **Single Point Crossover-** A random point is chosen on the individual chromosomes (strings) and the genetic material is exchanged at this point.

Chromosome1	11011 00100110110
Chromosome 2	11011 11000011110
Offspring 1	11011 11000011110
Offspring 2	11011 00100110110

Two point crossover

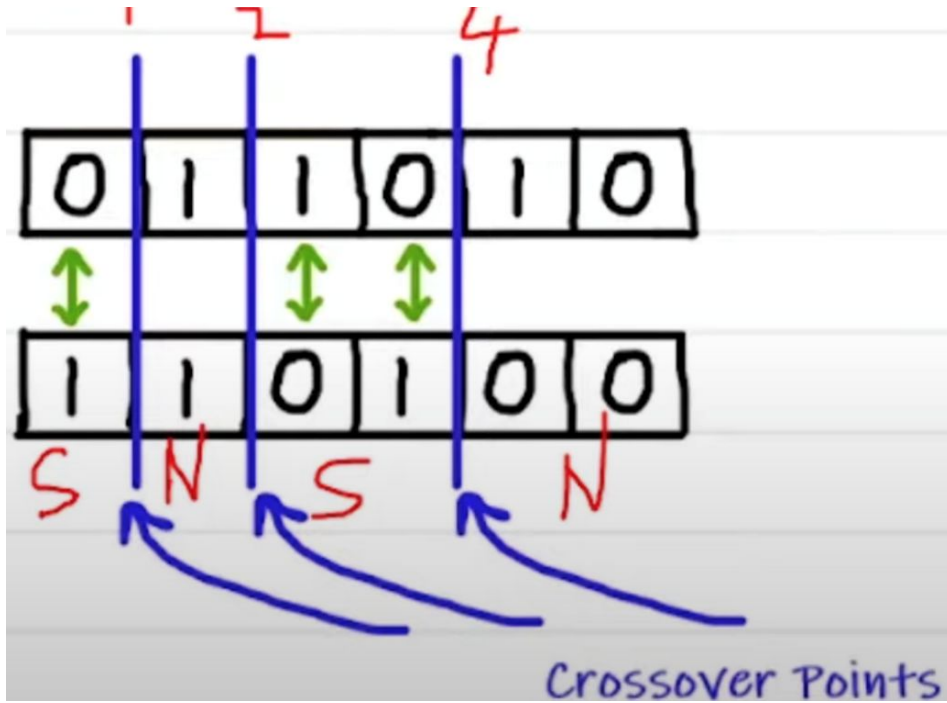


Offsprings

O₁: 0 1 0 1 1 0

O₂: 1 1 1 0 0 0

Multi-point crossover



Offsprings

O1: 1 1 0 1 1 0

O2: 0 1 1 0 0 0

Crossover

- **Two-Point Crossover-** Two random points are chosen on the individual chromosomes (strings) and the genetic material is exchanged at these points.

Chromosome1	11011 00100 110110
Chromosome 2	10101 11000 011110
Offspring 1	10101 00100 011110
Offspring 2	11011 11000 110110

NOTE: These chromosomes are different from the last example.

Crossover

- **Uniform Crossover-** Each gene (bit) is selected randomly from one of the corresponding genes of the parent chromosomes.

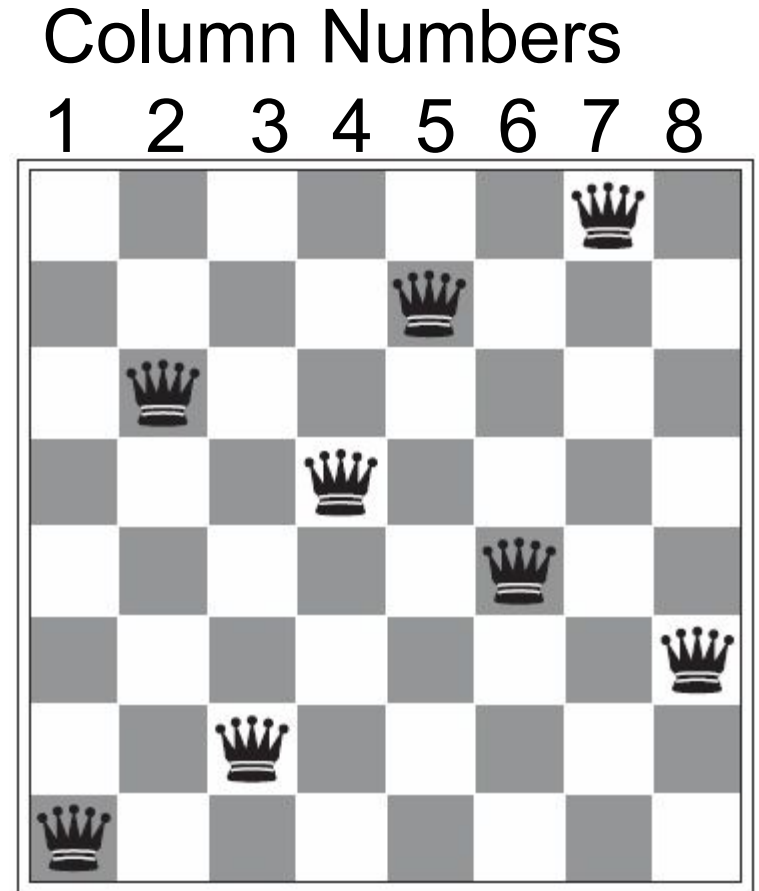
Chromosome1	11011 00100 110110
Chromosome 2	10101 11000 011110
Offspring	10111 00000 110110

NOTE: Uniform Crossover yields ONLY 1 offspring.

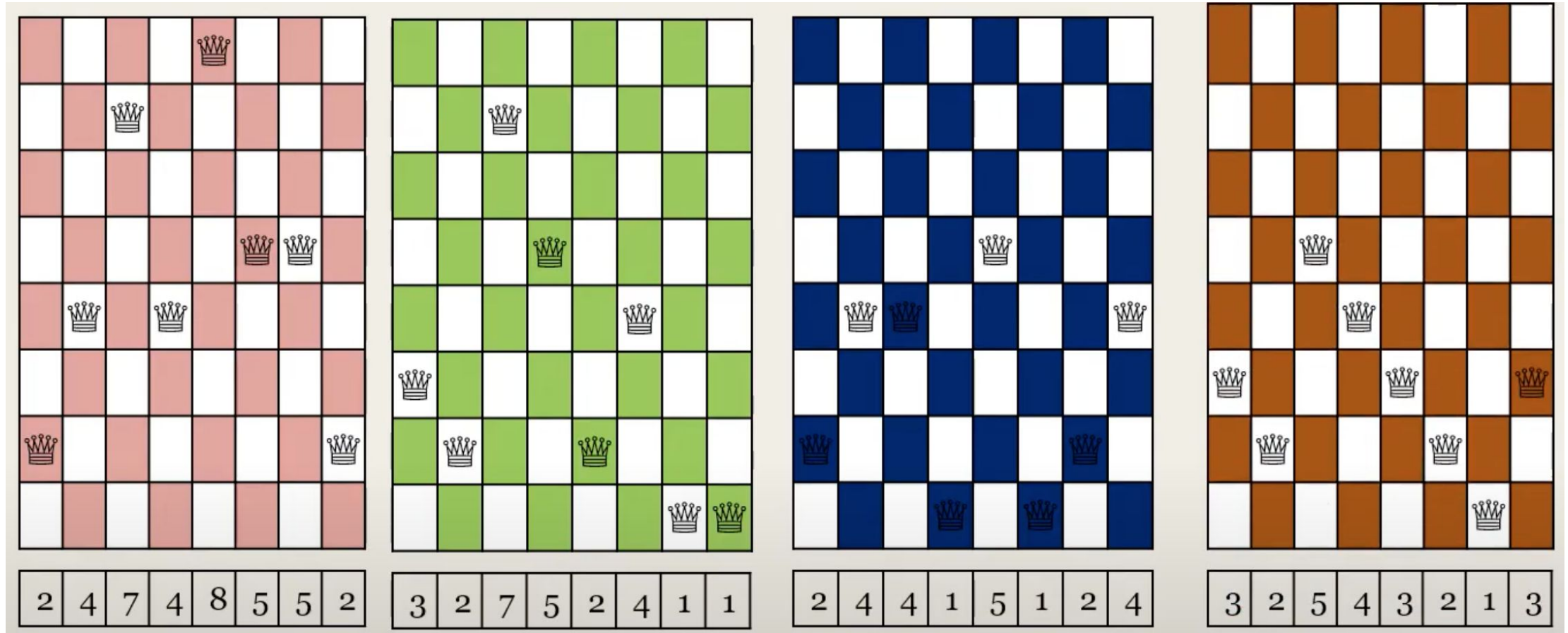
8 Queens Problem

Example: 8 Queens

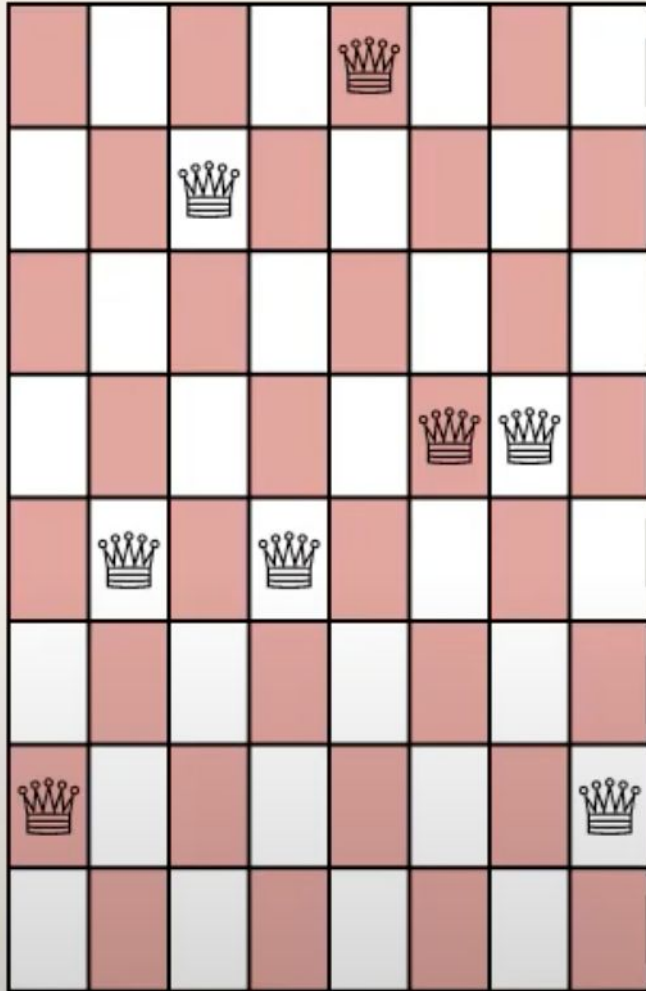
- **Population:**
 - Row of queen in each column.
 - [1,6,2,5,7,4,8,3]
- **Fitness Function**
 - Number of non-attacking pairs of queens



Initial Population



Calculating the Fitness value



2	4	7	4	8	5	5	2
---	---	---	---	---	---	---	---

Q1 :6

Q2 :5

Q3 :4

Q4 :4

Q5 :3

Q6 :1

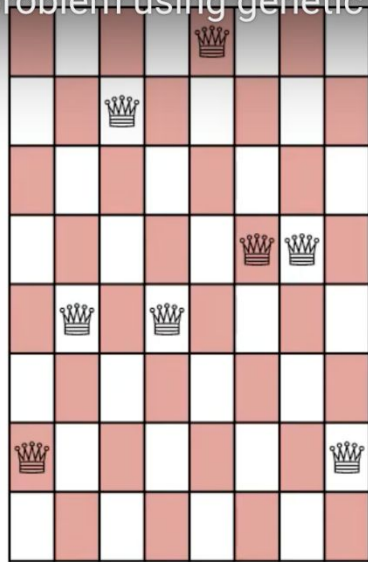
Q7 :1

Q8 :0

Fitness Value= 24

Calculating the Fitness value

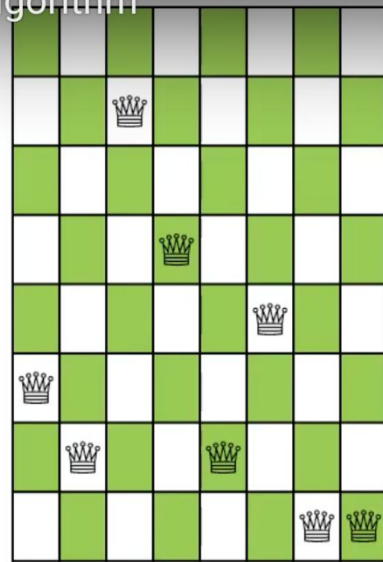
problem using genetic algorithm



2	4	7	4	8	5	5	2
---	---	---	---	---	---	---	---

Q1 :6
Q2 :5
Q3 :4
Q4 :4
Q5 :3
Q6 :1
Q7 :1
Q8 :0

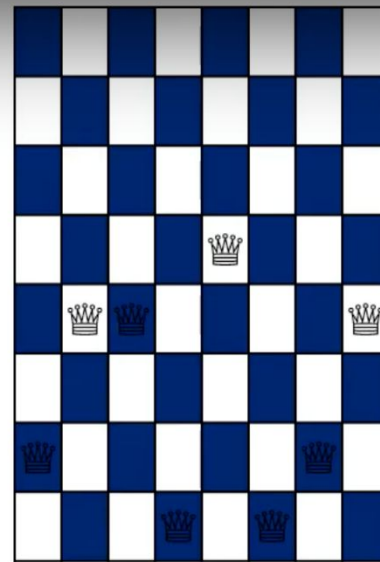
Fitness= 24



3	2	7	5	2	4	1	1
---	---	---	---	---	---	---	---

Q1 :6
Q2 :5
Q3 :4
Q4 :3
Q5 :3
Q6 :2
Q7 :0
Q8 :0

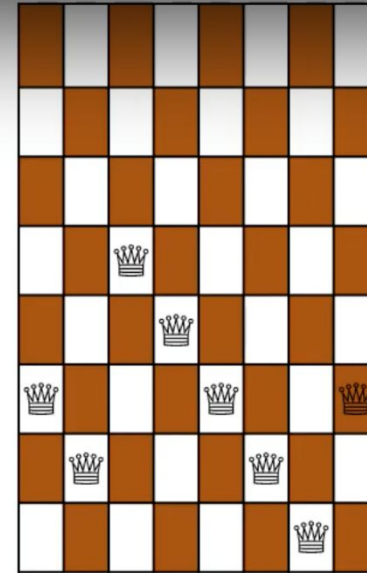
Fitness= 23



2	4	4	1	5	1	2	4
---	---	---	---	---	---	---	---

Q1 :5
Q2 :4
Q3 :3
Q4 :3
Q5 :3
Q6 :1
Q7 :1
Q8 :0

Fitness= 20



3	2	5	4	3	2	1	3
---	---	---	---	---	---	---	---

Q1 :3
Q2 :4
Q3 :1
Q4 :1
Q5 :0
Q6 :1
Q7 :1
Q1 :0

Fitness= 11

Fitness function

Represent states and compute fitness function.

24748552	24
32752411	23
24415124	20
32543213	11

(a)

Initial Population

Probability

Compute probability of being chosen (from fitness function).

24748552	24	31%
32752411	23	29%
24415124	20	26%
32543213	11	14%

(a)
Initial Population

Probability=

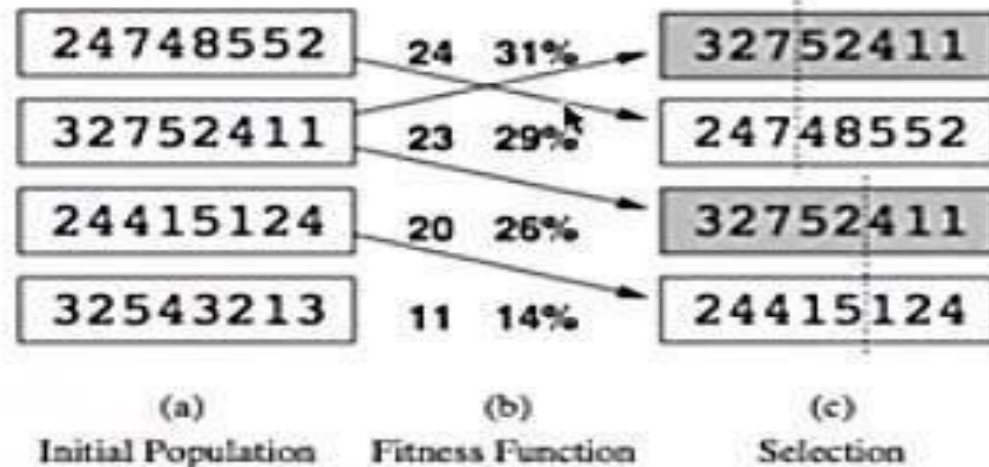
$$24+23+20+11= 78$$

$$24/78= 0.307 \text{ Normalize } 0.307 \times \mathbf{100} = \mathbf{30.7\%}$$

chance of being chosen probably

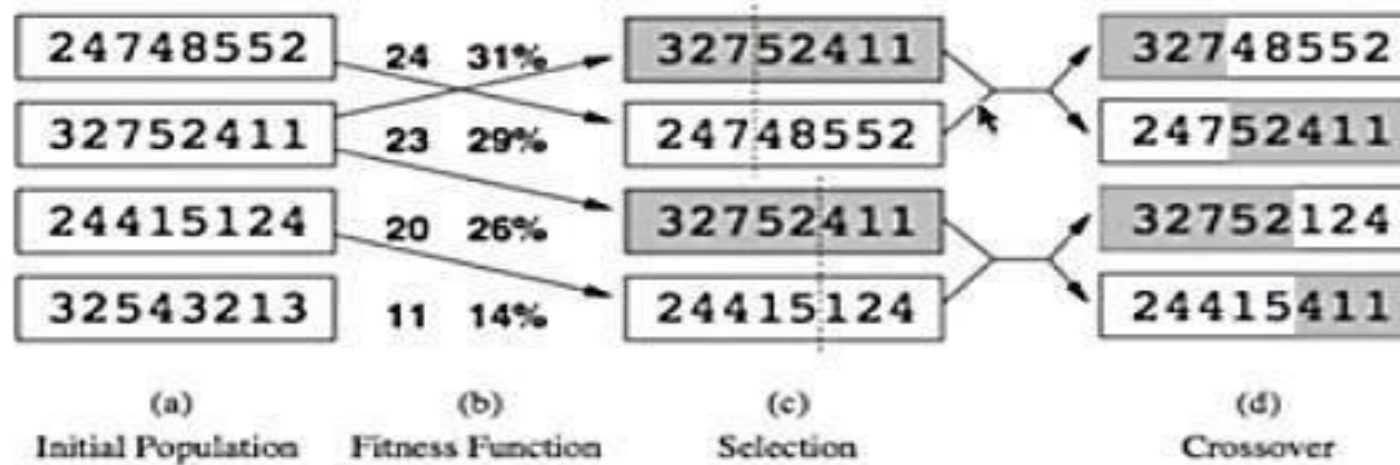
Reproduction

Randomly choose two pairs to reproduce based on probabilities. Pick a crossover point per pair.

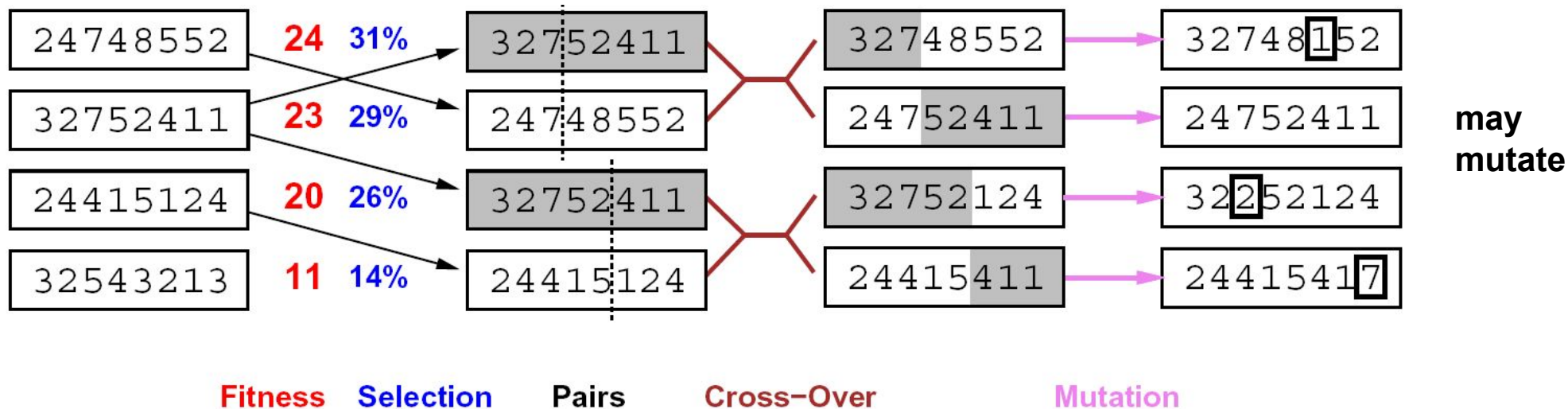


Crossover

Crossover, produce offspring.

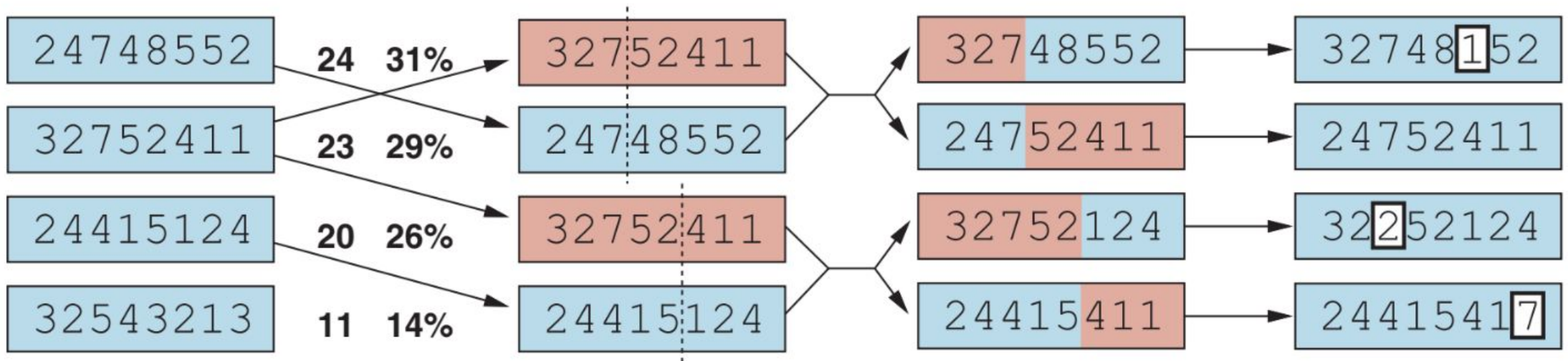


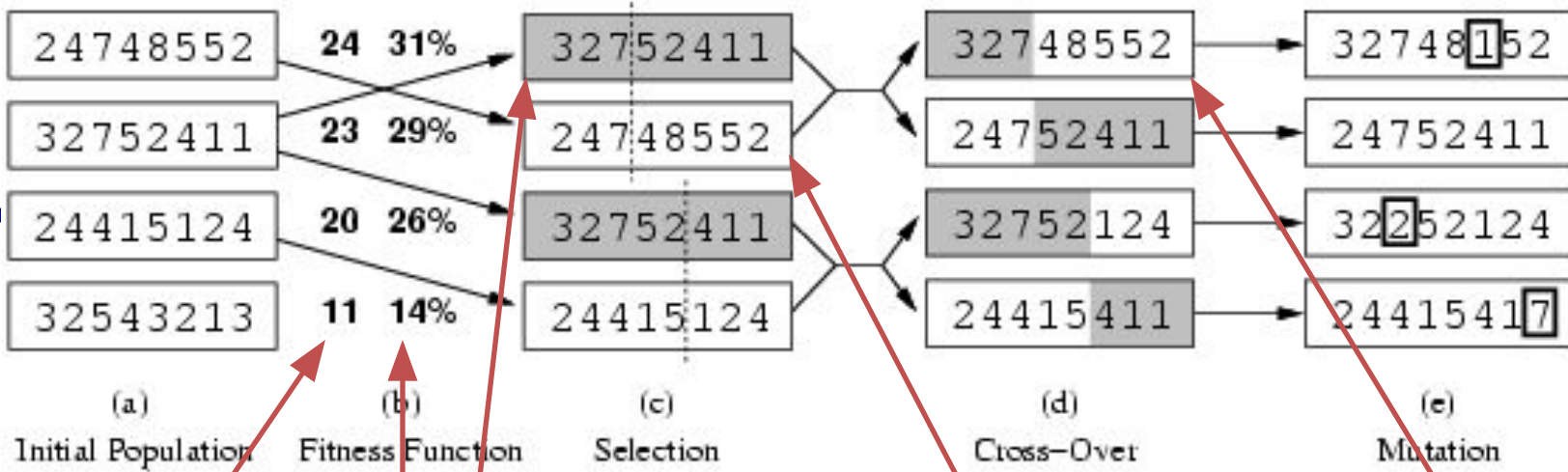
Mutation



- Genetic algorithms use a natural selection metaphor
 - Resample *K* individuals at each step (selection) weighted by fitness function
 - Combine by pairwise crossover operators, plus mutation to give variety

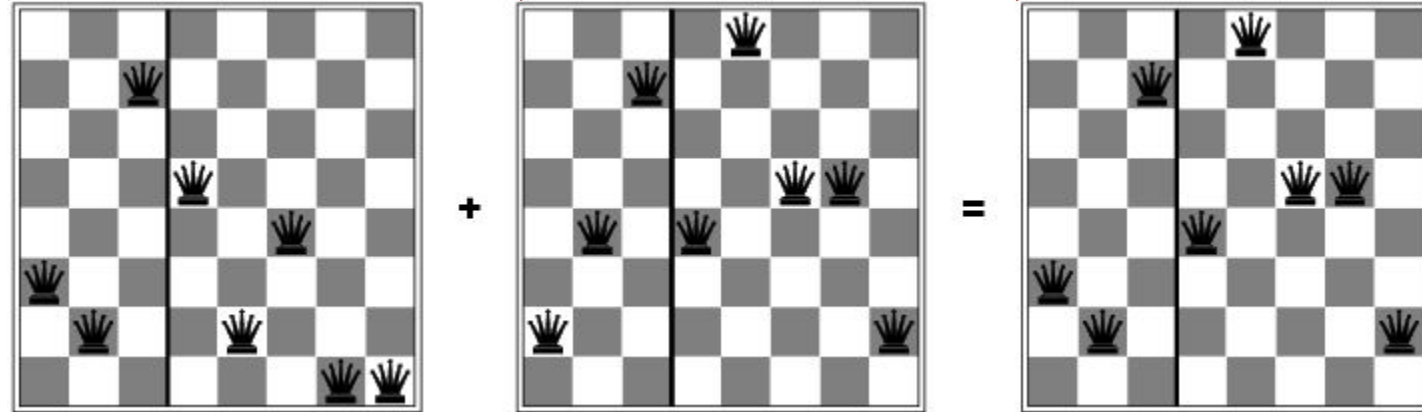
8 Queen Problem





fitness =
#non-attacking
queens

probability of being
in next generation =
 $\text{fitness} / (\sum_i \text{fitness}_i)$



- Fitness function: #non-attacking queen pairs
 - min = 0, max = $8 \times 7/2 = 28$
- $\sum_i \text{fitness}_i = 24 + 23 + 20 + 11 = 78$
- $P(\text{pick child}_1 \text{ for next gen.}) = \text{fitness}_1 / (\sum_i \text{fitness}_i) = 24/78 = 31\%$
- $P(\text{pick child}_2 \text{ for next gen.}) = \text{fitness}_2 / (\sum_i \text{fitness}_i) = 23/78 = 29\%$; etc

How to convert a
fitness value into a
probability of being in
the next generation.

Knapsack Problem

Knapsack Problem

- Let's say, you are going to spend a month in the wilderness. Only thing you are carrying is the backpack which can hold a maximum weight of 30 kg.
- Now you have different survival items, each having its own “Survival Points” (which are given for each item in the table).
- So, your objective is to maximize the survival points.
- Here is the table giving details about each item.

ITEM	WEIGHT	SURVIVAL POINTS
SLEEPING BAG	15	15
ROPE	3	7
POCKET KNIFE	2	10
TORCH	5	5
BOTTLE	9	8
GLUCOSE	20	17

Knapsack Problem

- Steps to follow:
 - Choose an initial population of four. Then choose best three subsequently (Encoding hint: 1 if item is chosen 0 otherwise)
 - Decide crossover point.
 - Chance of mutation = 50 items in 100 generations.
 - Step by step solution ahead

① Initial population (Initially I am taking 4 states chosen randomly which have weight ≤ 30)

100110
000011
011101
111100

② Second step : Selection / Fitness function : The one with highest survival point is to be selected (i.e. best 3 are the ones which have highest survival point).

100110	→	15+0+0+5+8+0 = 28 ✓
000011	fitness →	0+0+0+0+8+17 = 25
011101	function →	0+7+10+5+0+17 = 39 ✓
111100	→	15+7+10+5+0+0 = 37 ✓

Best three are states 1, 3 & 4

③ Selection : 1, 3 & 4 states

Combination : 1 & 3 & 3 & 4

<u>1 & 3</u>													
1	0	0	1	1	0		}	0	1	1	1	0	1
0	1	1	1	0	1		}	1	1	1	1	0	0

④ Enter

④ Now cross over will be applied.

1 2 3
 1 0 0 1 1 0
 0 1 1 1 0 1

3 2 4
 0 1 1 1 0 1
 1 1 1 1 0 0

Cross over decided after 3 digits.

1 0 0 1 0 1
 0 1 1 1 1 0

0 1 1 1 0 0
 1 1 1 1 0 1

⑤ Now mutation (we had 50 items in 100 generations).

Because in question we are told to perform 3 iterations; one of them is initialization iteration & 2 are generation thereby

$$\frac{50 \text{ items}}{n} \times \frac{100 \text{ generation}}{2} = 1$$

Thereby one bit mutation will only take place in this entire 3 iteration phase.

Our cross over brought 4 new states.

1 0 0 1 0 (1) mutate to 0.
0 1 1 1 1 0
0 1 1 1 0 0
1 1 1 1 0 1

Now if you want you can mutate one bit in this iteration, else do it in any ~~to~~ other iteration. I have mutated in this iteration thereby output of first iteration is

1	0	0	1	0	0
0	1	1	1	1	0
0	1	1	1	0	0
1	1	1	1	0	1

Now this output of first iteration will be the input for second iteration.

① Second iteration

① Initialization: Previous iteration output.

1	0	0	1	0	0
0	1	1	1	1	0
0	1	1	1	0	0
1	1	1	1	0	1

② Fitness function:

1	0	0	1	0	0	→	15 + 0 + 0 + 5 + 0 + 0 = 20
0	1	1	1	1	0	→	0 + 7 + 10 + 5 + 8 + 0 = 25 =
0	1	1	1	0	0	→	0 + 7 + 10 + 5 + 0 + 0 = 22 =
1	1	1	1	0	1	→	15 + 7 + 10 + 5 + 0 + 17 = 54 =

③ Selection 2 & 3 & 4

④ Crossover

2 & 3

0	1	1	1	1	0
0	1	1	1	0	0

0	1	1	1	0	0
0	1	1	1	1	0

3 & 4

0	1	1	1	0	0
1	1	1	1	0	1

0	1	1	1	0	1
1	1	1	1	0	0

Crossover results in

0	1	1	1	0	0
0	1	1	1	1	0
0	1	1	1	0	1
1	1	1	1	0	0

No mutation now because I have already done in previous iteration & only one bit mutation was allowed in the question

* Point to notice: See state 3 in this crossover that's exactly holding weight 30 & having survival 39 which is till now the best state as we can also hold more items & survival is also more.

Now as no mutation is done we will input the crossover output of this iteration to third iteration.

That mean.

③ Third iteration

Initialization

0	1	1	1	0	0
0	1	1	1	1	0
0	1	1	1	0	1
1	1	1	1	0	0

→ Now rest of the steps will be done & the end.

You can do the third iteration on your own now. First two iterations have been done by me.

Properties

- ▶ Work well for mixed (continuous and discrete) problems
- ▶ They are less susceptible to get stuck at local optima
- ▶ Computationally expensive
- ▶ However, easy to perform in parallel
- ▶ No math in the process. The objective (fitness) function may be hard

Genetic Algorithms

A genetic algorithm: each mating of two parents produces only one offspring, not two.

function GENETIC-ALGORITHM(*population*, FITNESS-FN) **returns** an individual

inputs: *population*, a set of individuals

 FITNESS-FN, a function that measures the fitness of an individual

repeat

new_population $\leftarrow$ empty set

for $i = 1$ **to** SIZE(*population*) **do**

$x \leftarrow$ RANDOM-SELECTION(*population*, FITNESS-FN)

$y \leftarrow$ RANDOM-SELECTION(*population*, FITNESS-FN)

$child \leftarrow$ REPRODUCE(x, y)

if (small random probability) **then** $child \leftarrow$ MUTATE($child$)

 add $child$ to *new_population*

population $\leftarrow$ *new_population*

until some individual is fit enough, or enough time has elapsed

return the best individual in *population*, according to FITNESS-FN

function REPRODUCE(x, y) **returns** an individual

inputs: x, y , parent individuals

$n \leftarrow$ LENGTH(x); $c \leftarrow$ random number from 1 to n

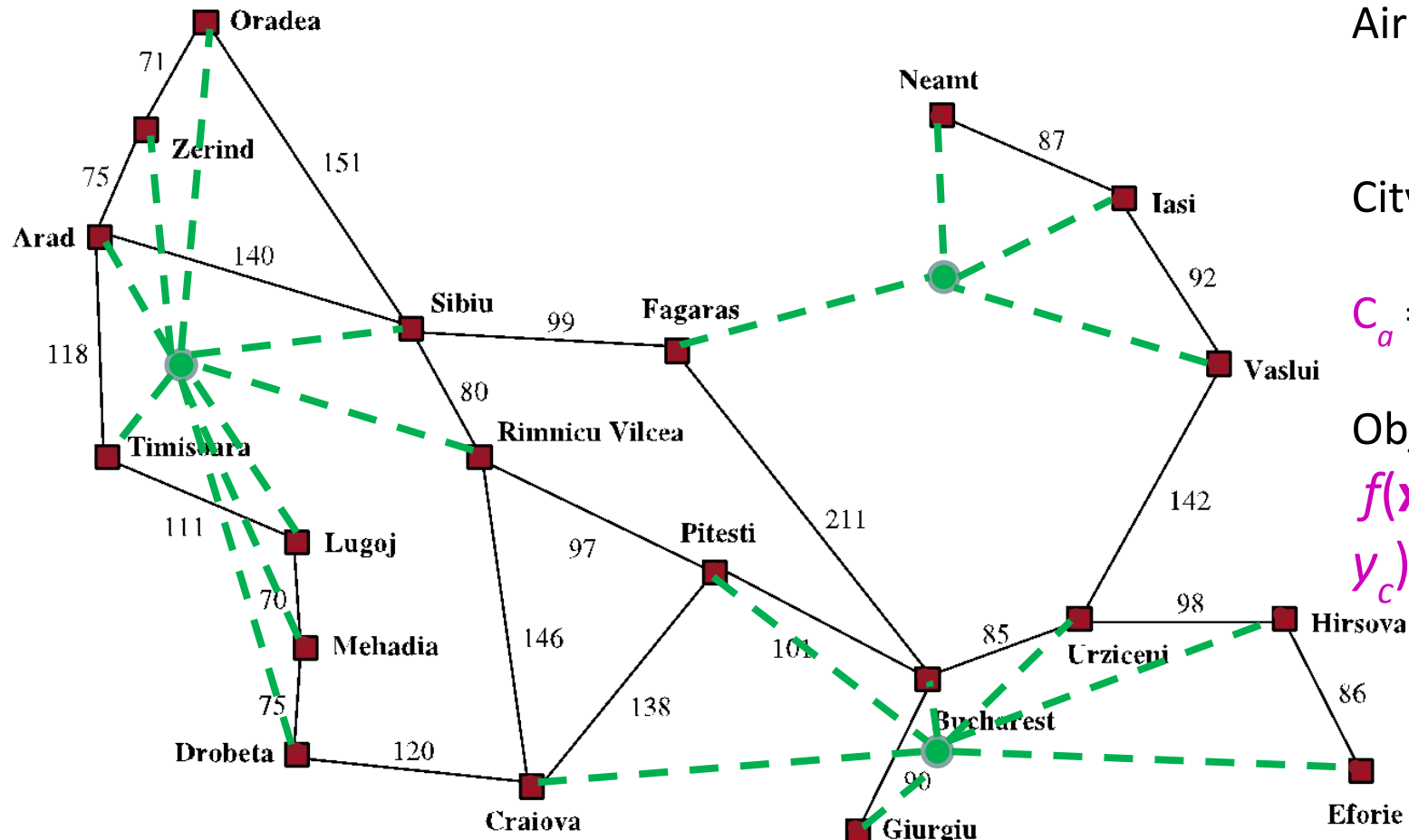
return APPEND(SUBSTRING($x, 1, c$), SUBSTRING($y, c + 1, n$))

Local search in continuous spaces



Example: Placing airports in Romania

Place 3 airports to minimize the sum of squared distances from each city to its nearest airport



Airport locations

$$\mathbf{x} = (x_1, y_1), (x_2, y_2), (x_3, y_3)$$

City locations (x_c, y_c)

C_a = cities closest to airport a

Objective: minimize

$$f(\mathbf{x}) = \sum_a \sum_{c \in C_a} (x_a - x_c)^2 + (y_a - y_c)^2$$

Handling a continuous state/action space

1. Discretize it!

- Define a grid with increment δ , use any of the discrete algorithms

2. Choose random perturbations to the state

- a. First-choice hill-climbing: keep trying until something improves the state
- b. Simulated annealing

4. Compute gradient of $f(\mathbf{x})$ analytically

Finding extrema in continuous space

- Gradient vector $\nabla f(\mathbf{x}) = (\partial f / \partial x_1, \partial f / \partial y_1, \partial f / \partial x_2, \dots)^\top$
- For the airports, $f(\mathbf{x}) = \sum_a \sum_{c \in C_a} (x_a - x_c)^2 + (y_a - y_c)^2$
- $\partial f / \partial x_1 = \sum_{c \in C_1} 2(x_1 - x_c)$
- At an extremum, $\nabla f(\mathbf{x}) = 0$
- Can sometimes solve in closed form: $x_1 = (\sum_{c \in C_1} x_c) / |C_1|$
 - Is this a local or global minimum of f ?
- If we can't solve $\nabla f(\mathbf{x}) = 0$ in closed form...
 - Gradient descent: $\mathbf{x} \leftarrow \mathbf{x} - \alpha \nabla f(\mathbf{x})$
- Huge range of algorithms for finding extrema using gradients